

Compte rendu de TP Firebase — Firestore et Authentication

Étudiant Délio Duchesne-Brajon — Groupe 1
Bloc Bloc 2 — Développement
Outils Flutter · Cloud Firestore · Firebase Auth · Supabase
Date 2 octobre 2026

SOMMAIRE

1. Objectif du TP	2
2. Cloud Firestore	2
2.1. Première collection	2
2.2. Collection items	3
3. Application Flutter — Firebase Todolist.....	4
3.1. Écriture depuis l'application.....	4
3.2. Lecture et affichage	5
3.3. Structure du code	5
3.4. Liste de tâches complète	6
4. Firebase Authentication.....	6
4.1. Page de connexion.....	6
4.2. Page de profil	7
4.3. Inscription d'un second compte	8
5. Profil utilisateur dans Firestore.....	9
6. Exercices — Liaison avec Supabase	10
6.1. Exercice 4.1 — Informations de connexion	10
6.2. Exercice 4.2 — Ajout de la dépendance.....	11
7. Conclusion.....	11

1. Objectif du TP

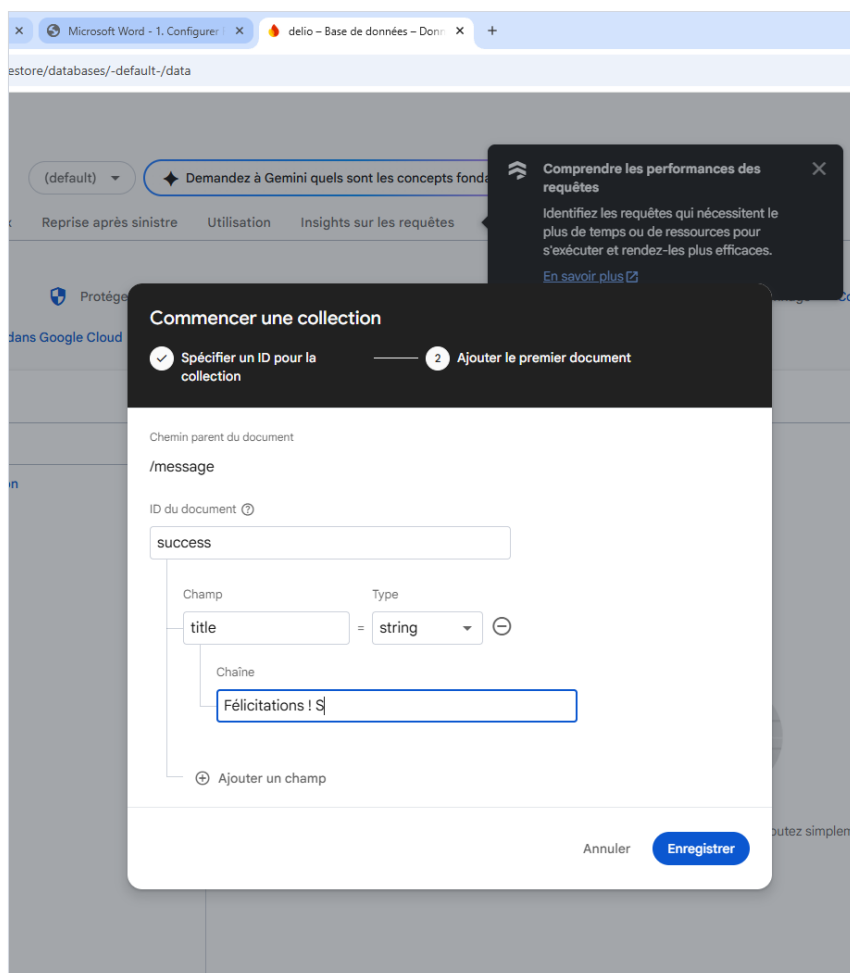
Utiliser Firebase comme back-end d'une application Flutter : structurer une base Cloud Firestore, y lire et y écrire depuis l'application, puis gérer les comptes utilisateurs avec Firebase Authentication.

L'application développée est une liste de tâches, « Firebase Todolist », exécutée dans Chrome. Les exercices de fin de TP préparent la même liaison avec Supabase.

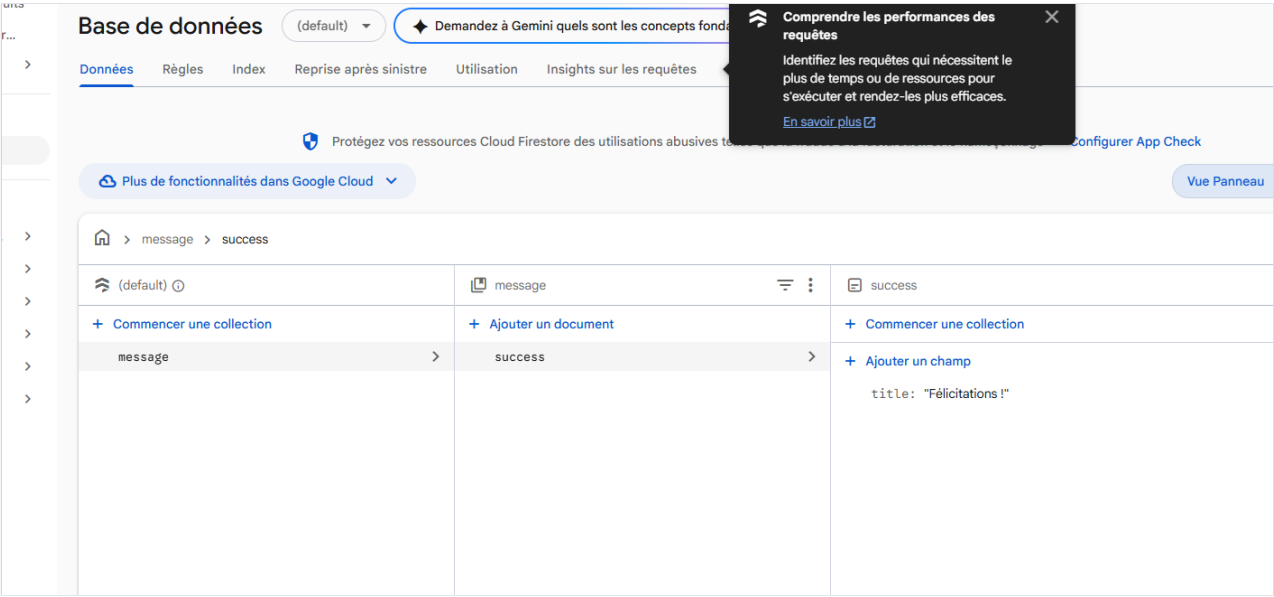
2. Cloud Firestore

2.1. Première collection

Une collection message est créée dans la console Firebase, avec un premier document success contenant un champ title de type *string*.

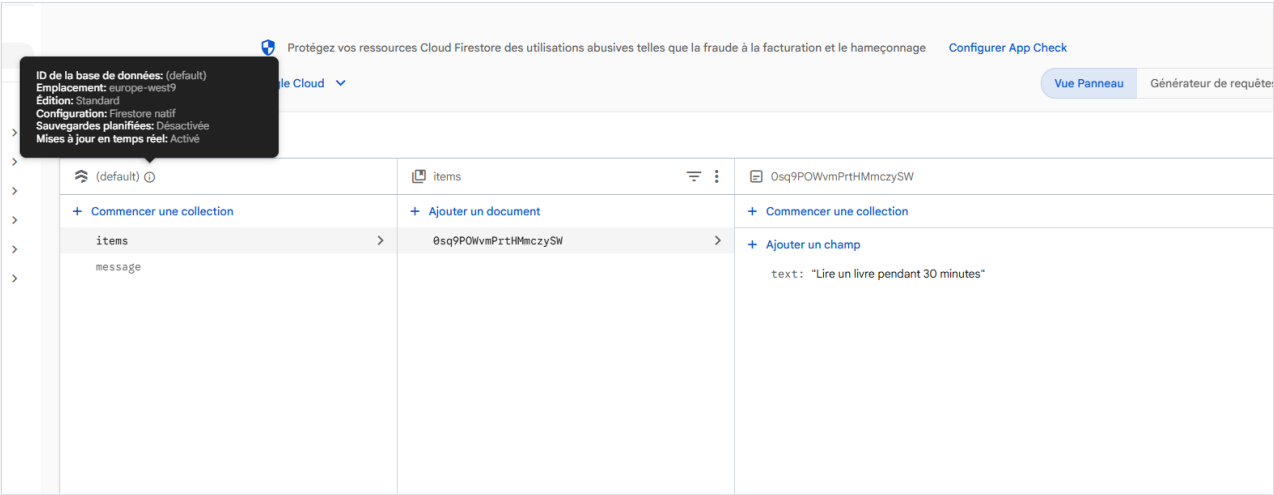


Le document apparaît dans la vue en trois colonnes : la collection, le document, puis ses champs.



2.2. Collection items

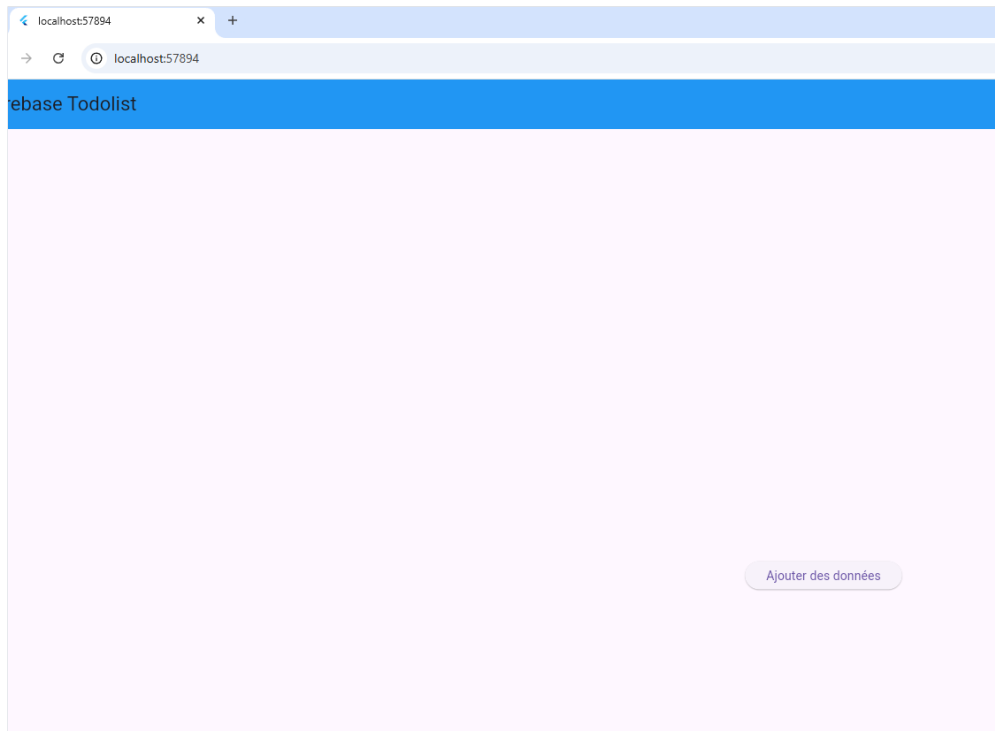
Une seconde collection, items, reçoit les tâches de la liste. Chaque document possède un identifiant généré automatiquement et un champ text.



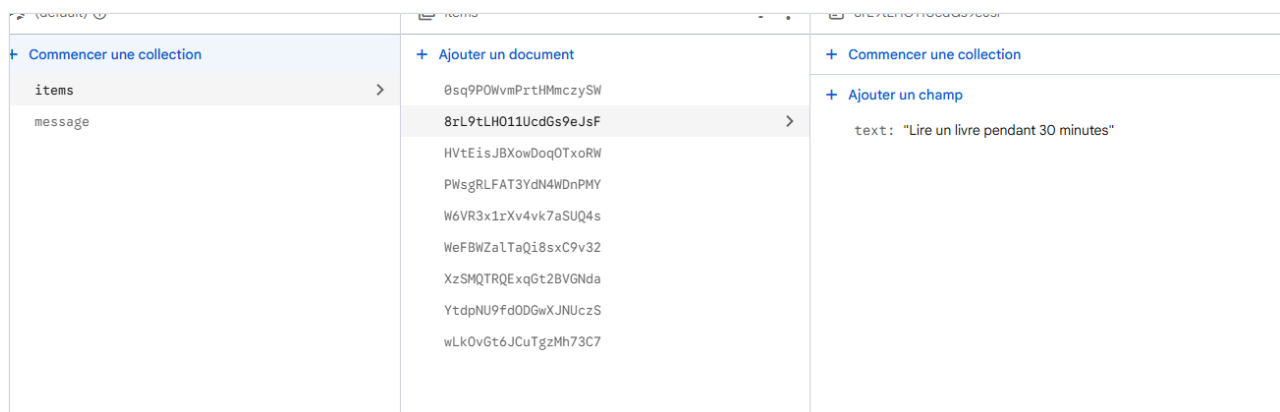
3. Application Flutter — Firebase Todolist

3.1. Écriture depuis l'application

L'application comporte une barre de titre et un bouton « Ajouter des données », qui écrit un document dans la collection `items`.

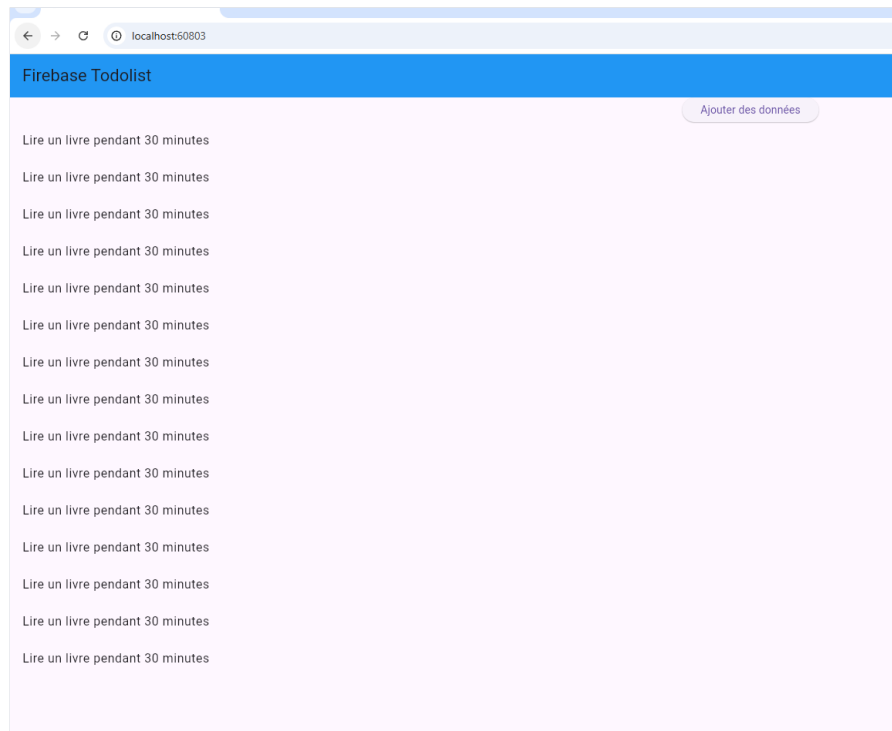


Chaque appui crée un nouveau document, ce que confirme la console.



3.2. Lecture et affichage

L'application lit ensuite la collection et affiche une ligne par document. La liaison entre Flutter et Firestore fonctionne donc dans les deux sens.



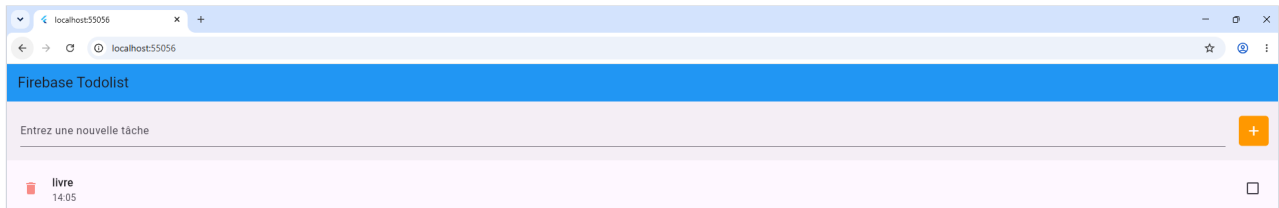
3.3. Structure du code

L'interface est construite dans un Scaffold : une AppBar pour le titre, un ElevatedButton qui appelle `addDataToFirebase()` et un Expanded contenant le widget `ListSection()`, chargé de l'affichage.

```
74 final databaseReference = FirebaseFirestore.instance;
75
76 @override
77 Widget build(BuildContext context) {
78   return MaterialApp(
79     debugShowCheckedModeBanner: false,
80     theme: ThemeData(primarySwatch: Colors.amber),
81     home: Scaffold(
82       appBar: AppBar(
83         title: const Text('Firebase Todolist'),
84         backgroundColor: Colors.blue,
85       ), // AppBar
86       body: Column(
87         children: [
88           Center(
89             child: ElevatedButton(
90               onPressed: () {
91                 addDataToFirebase();
92               },
93               child: const Text('Ajouter des données'),
94             ), // ElevatedButton
95           ), // Center
96           Expanded(
97             child: ListSection(),
98           ), // Expanded
99         ],
100       ), // Column
101     ), // Scaffold
102   ); // MaterialApp
103 }
```

3.4. Liste de tâches complète

Le bouton de test est remplacé par un champ de saisie. Chaque tâche est affichée avec son heure de création, une case à cocher et une icône de suppression.



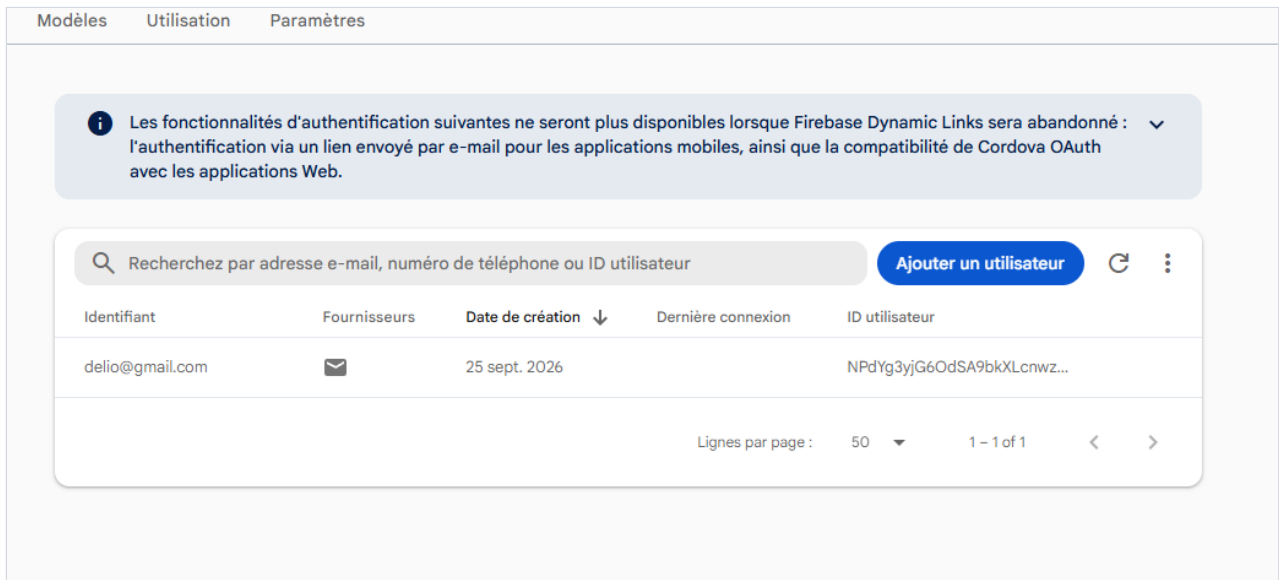
4. Firebase Authentication

4.1. Page de connexion

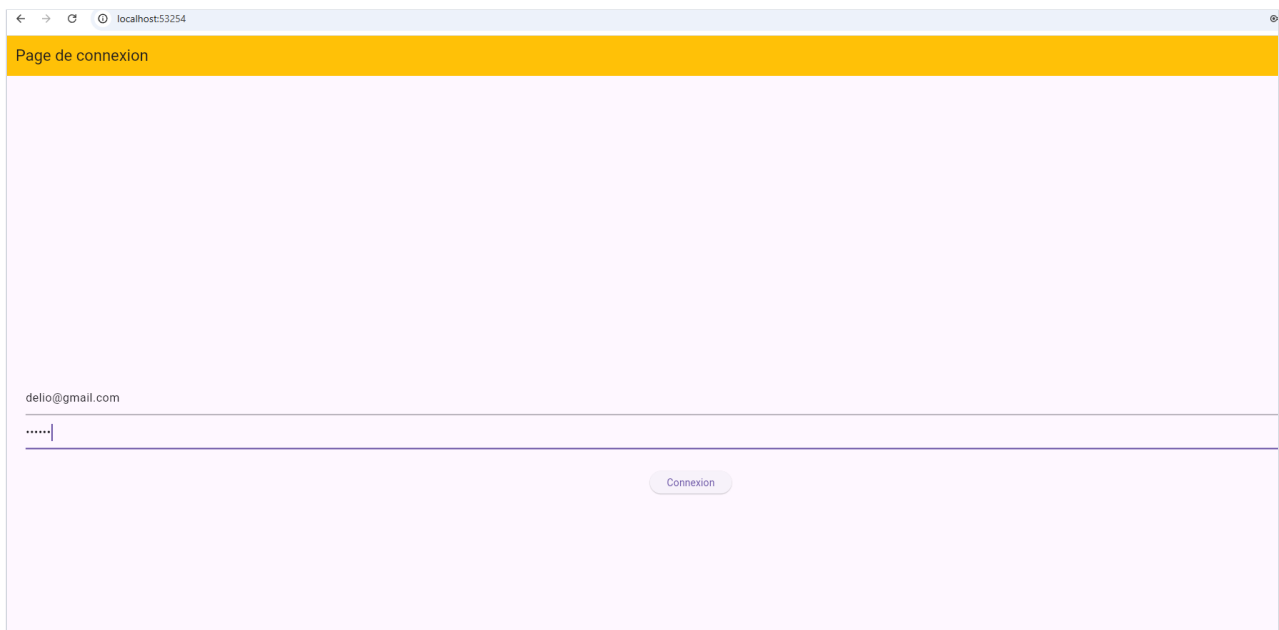
Une page de connexion est ajoutée à l'application.



Un premier compte est créé dans l'onglet Authentication de la console.



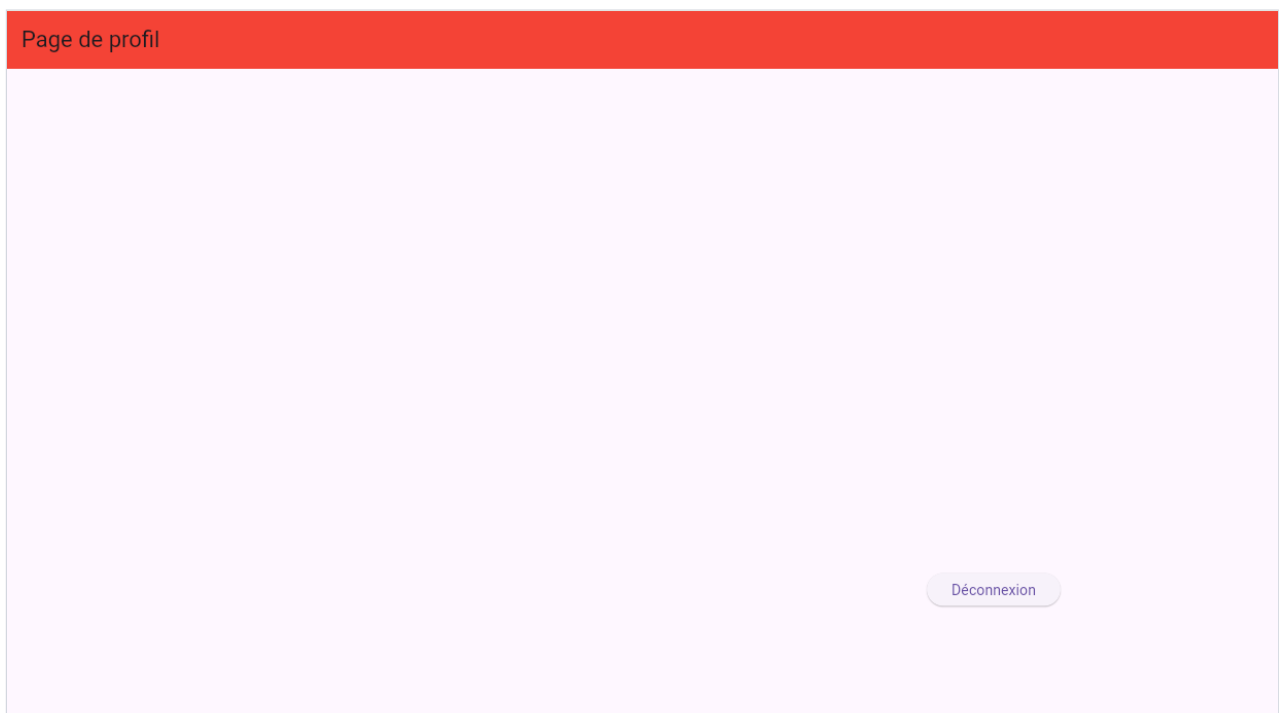
L'adresse et le mot de passe sont ensuite saisis dans l'application.



The screenshot shows a web browser window with the address bar displaying 'localhost:53254'. The page has a yellow header bar with the text 'Page de connexion'. Below the header, there is a large, empty white rectangular area. At the bottom of this area, there is a text input field containing the email address 'delio@gmail.com'. Below the email field, there is a password input field with a masked password '.....'. To the right of the password field, there is a button labeled 'Connexion'.

4.2. Page de profil

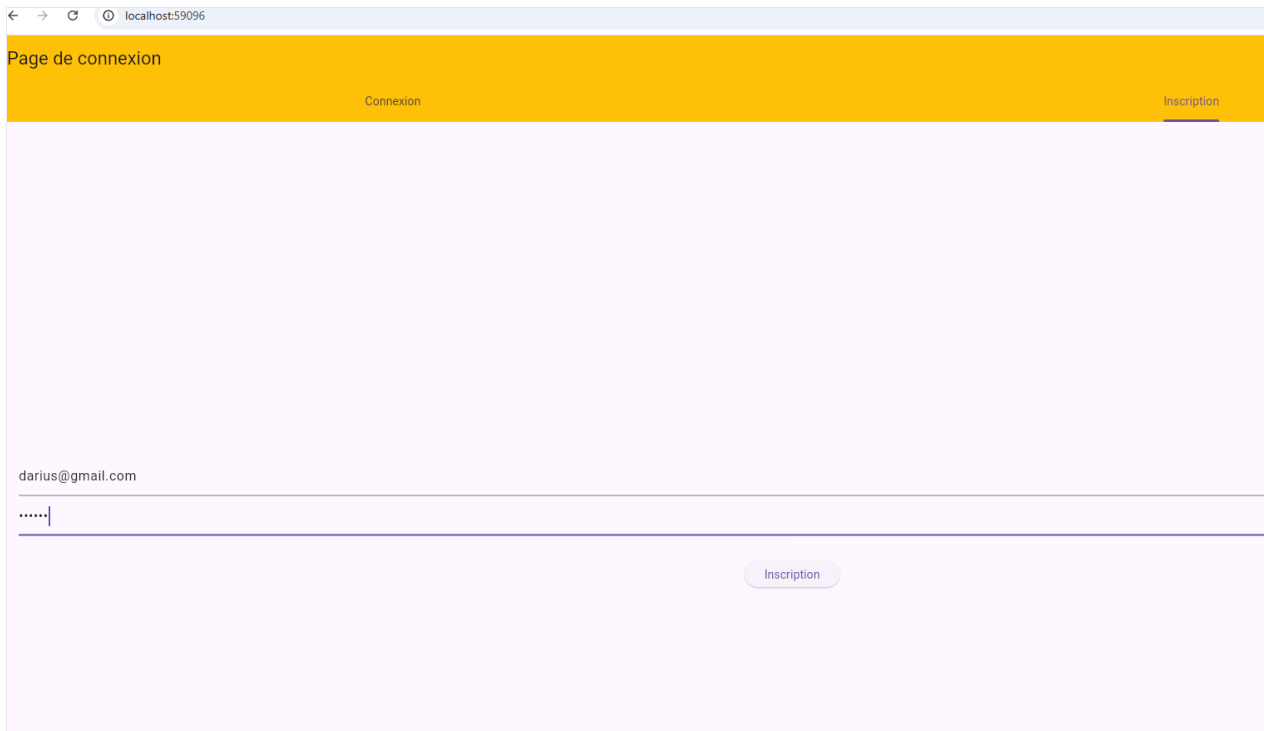
Une fois authentifié, l'utilisateur arrive sur une page de profil munie d'un bouton de déconnexion.



The screenshot shows a web browser window with a red header bar containing the text 'Page de profil'. Below the header, there is a large, empty white rectangular area. In the bottom right corner of this area, there is a button labeled 'Déconnexion'.

4.3. Inscription d'un second compte

La page propose aussi un onglet « Inscription », utilisé pour créer un second compte depuis l'application.



Page de connexion

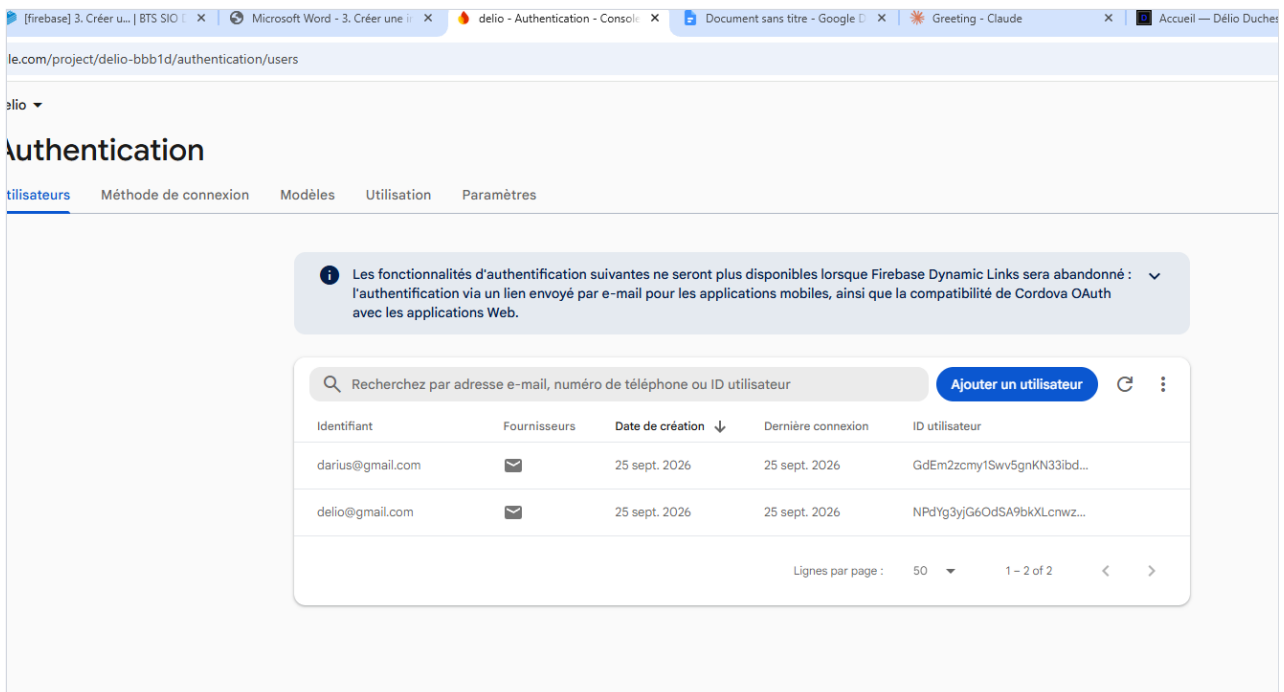
Connexion Inscription

darius@gmail.com

.....

Inscription

Le compte apparaît immédiatement dans la console, qui compte alors deux utilisateurs.



le.com/project/delio-bbb1d/authentication/users

Authentication

Utilisateurs Méthode de connexion Modèles Utilisation Paramètres

Les fonctionnalités d'authentification suivantes ne seront plus disponibles lorsque Firebase Dynamic Links sera abandonné : l'authentification via un lien envoyé par e-mail pour les applications mobiles, ainsi que la compatibilité de Cordova OAuth avec les applications Web.

Recherchez par adresse e-mail, numéro de téléphone ou ID utilisateur

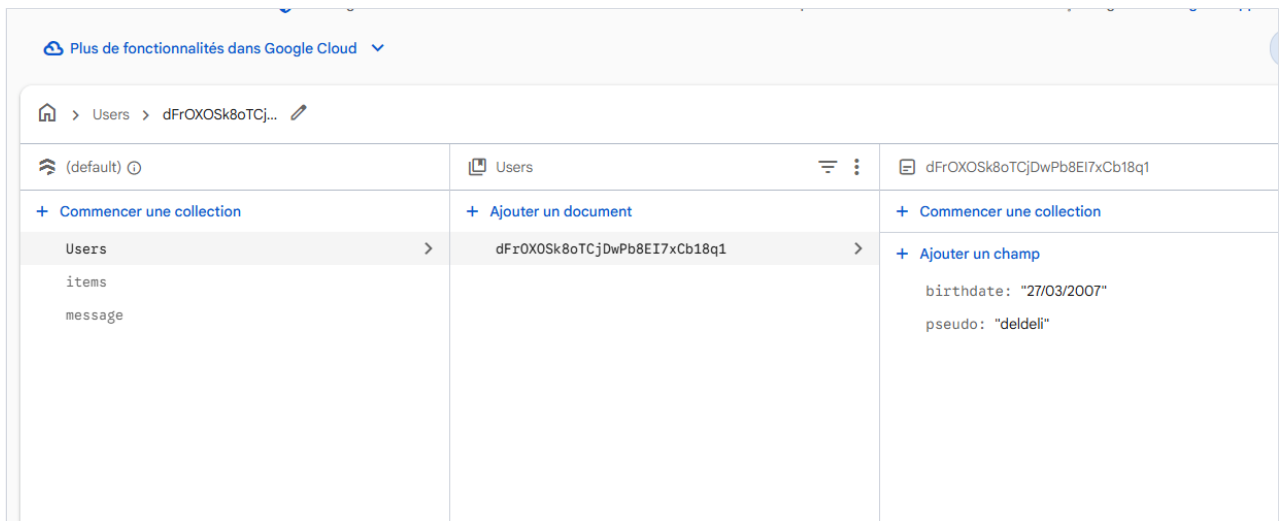
Ajouter un utilisateur

Identifiant	Fournisseurs	Date de création ↓	Dernière connexion	ID utilisateur
darius@gmail.com	✉	25 sept. 2026	25 sept. 2026	GdEm2zcmy1Swv5gnKN33ibd...
delio@gmail.com	✉	25 sept. 2026	25 sept. 2026	NPdYg3yjG6OdSA9bkXLcnwz...

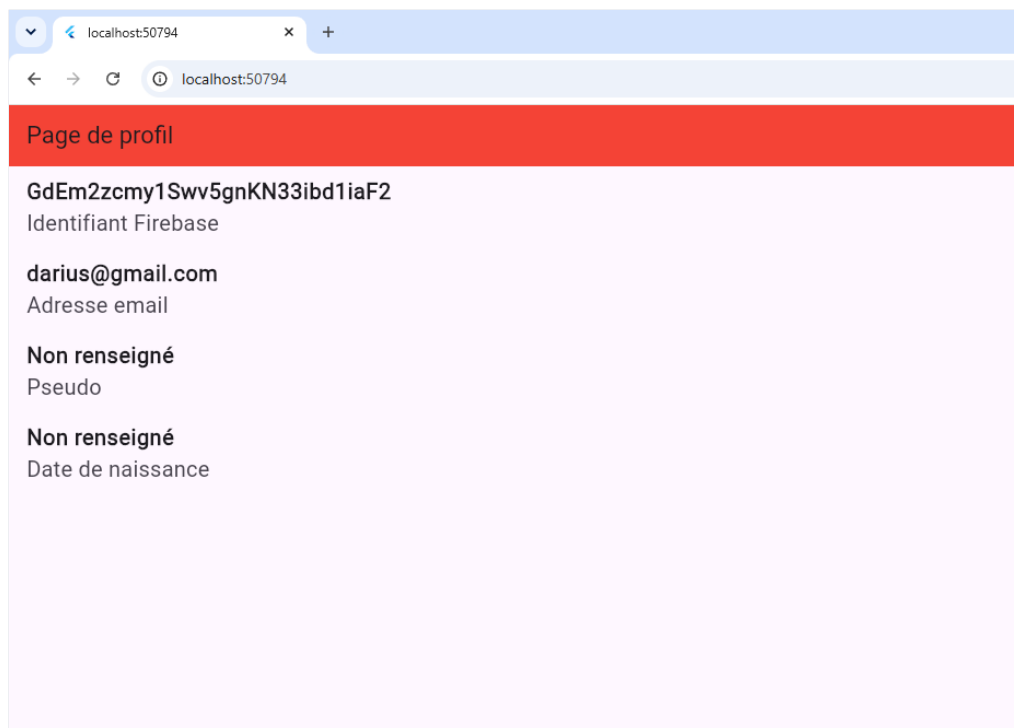
Lignes par page : 50 1 - 2 of 2

5. Profil utilisateur dans Firestore

Une troisième collection, Users, stocke les informations que l'authentification ne gère pas : un pseudo et une date de naissance.



La page de profil affiche bien l'identifiant Firebase et l'adresse e-mail du compte connecté, mais indique « Non renseigné » pour le pseudo et la date de naissance.

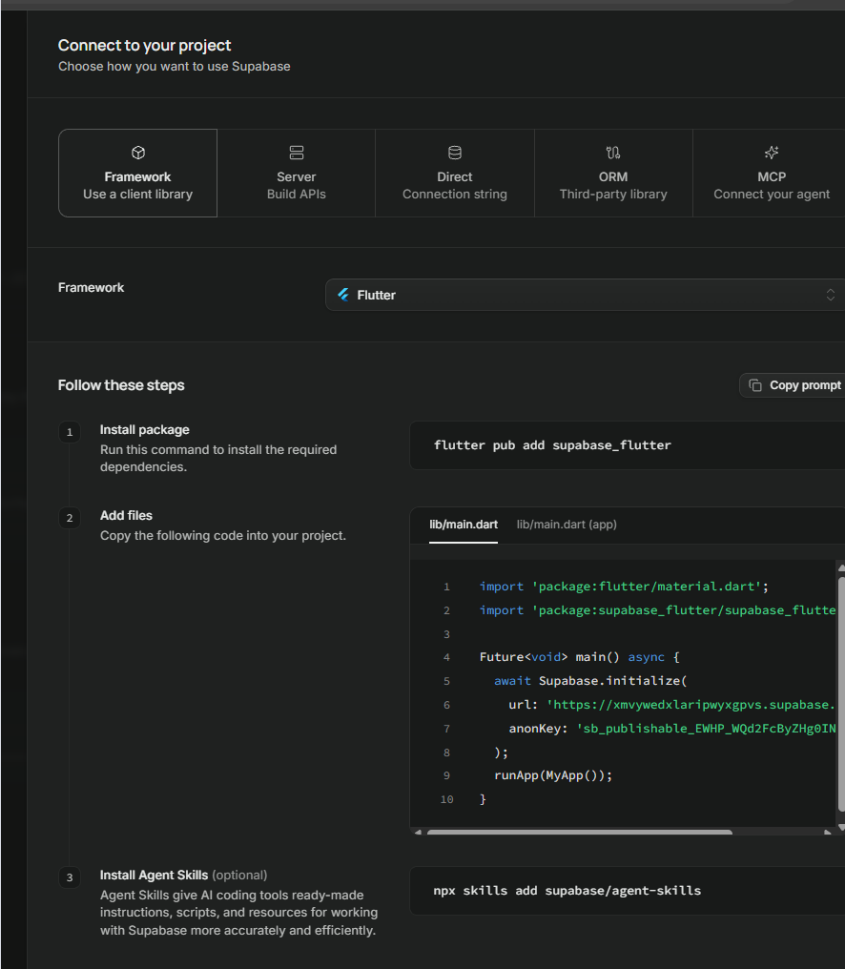


L'explication se trouve dans les identifiants : le document de Users porte un identifiant généré automatiquement, alors que la page recherche un document dont l'identifiant est l'UID du compte connecté. Pour que le profil se remplisse, le document doit être créé avec cet UID comme identifiant.

6. Exercices — Liaison avec Supabase

6.1. Exercice 4.1 — Informations de connexion

La console Supabase fournit, pour Flutter, la commande d'installation du paquet et le code d'initialisation à placer dans `main.dart`, avec l'URL du projet et sa clé publique.



The screenshot shows the 'Connect to your project' interface in the Supabase console. It offers five options: Framework (selected), Server, Direct, ORM, and MCP. Under the 'Framework' section, 'Flutter' is selected. The 'Follow these steps' section provides a three-step guide: 1. Install package (command: `flutter pub add supabase_flutter`), 2. Add files (code for `lib/main.dart`), and 3. Install Agent Skills (optional). The code for step 2 is as follows:

```
lib/main.dart lib/main.dart (app)
1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter';
3
4 Future<void> main() async {
5   await Supabase.initialize(
6     url: 'https://xmvywedxlaripwypgvs.supabase.',
7     anonKey: 'sb_publishable_EWHP_WQd2FcByZHg0IN',
8   );
9   runApp(MyApp());
10 }
```

Step 3 includes the command `npx skills add supabase/agent-skills`.

6.2. Exercice 4.2 — Ajout de la dépendance

La dépendance `supabase_flutter` est ajoutée au fichier `pubspec.yaml`, à la suite des paquets Firebase déjà présents.

```
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  english_words: ^4.0.0 # ma nouvelle dépendance
  google_fonts: ^8.2.1
  animate_do: ^5.1.0
  adaptive_theme: 3.7.2
  firebase_core: ^4.15.0
  cloud_firestore: ^6.10.0
  intl: ^0.20.3
  firebase_auth: ^6.7.0
  supabase_flutter: ^2.6.0
```

7. Conclusion

Ce TP couvre l'essentiel de Firebase côté application : une base Firestore organisée en collections et en documents, la lecture et l'écriture depuis Flutter, puis l'authentification par adresse e-mail et mot de passe.

Le point à retenir concerne le profil : les données complémentaires d'un utilisateur doivent être rangées dans un document portant son UID, sinon l'application ne peut pas les retrouver.